

《内容保真度与治理质量评估智能体（LLM-as-Judge）》 开题报告

一、人员分工与责任信息

本课题是一个系统性强、技术壁垒高、涉及算法与工程深度融合的综合性项目。为了保证项目的顺利推进与高质量交付，团队共由 3 名成员组成，实施“组长核心负责制”。组长承担了系统底层架构、文本对齐引擎以及并发多裁判共识核心算法的设计与研发，贡献度占比达 50%。

1.1 团队角色、姓名与贡献比例

- 组长：**李首澎 —— 负责系统架构、底层对齐引擎开发、多裁判并发投票共识服务及系统集成。
- 组员：**阿思晗 —— 负责大模型 Prompt 调试、LangChain4j 接口定义、自校准数学模型开发与测试集标注。
- 组员：**由靖喆 —— 负责前端质检看板开发、系统 API 联调测试及技术文档整理。

1.2 详细分工与责任矩阵说明

为保障项目高质量推进，团队细化了具体的任务包与责任边界，拒绝模糊分工，确保各模块职责清晰：

1. 李首澎—— 研发任务包

作为团队组长，全面负责系统的底层架构、核心对齐与决策算法的研发工作，是项目技术指标达标的第一责任人。具体任务包包括：

- 基础架构搭建：**基于 Spring Boot 3.x 搭建微服务框架，设计高内聚低耦合的业务包结构；设计前后端统一的数据传输模型（JSON Schema）与标准 RESTful 接口规范。
- 审计留痕设计：**规划全局统一的 `trace_id` 传递与审计回放机制，编写 Spring 拦截器拦截每一次评测请求，记录其输入输出，确保质检过程全程留痕、可追溯。
- 句级差分对齐引擎（MyersTextAligner）：**负责引入 `java-diff-utils` 库，设计中文分句正则，实现原文与治理文本的 Myers 差分比对算法，精准找出并对齐发生删除和改写的句子，为大模型裁判精准指引“行级可疑目标”。
- 多裁判并发决策引擎（VotingStableEvaluator）：**利用 `CompletableFuture` 编排多线程，并行运行多轮大模型判罚；编写方差控制与分数融合算法，实现多数票共识决策（Majority Voting）逻辑，去除单次调用的模型幻觉。
- 系统级联调集成：**负责前后端核心模块的联调与系统集成测试。

2. 阿思晗—— 研发任务包

主要负责大模型策略优化、评测数学指标的设计与自校准模块开发。具体任务包包括：

- AI 接口声明与 Prompt 工程：**基于 LangChain4j 的 `AiServices` 编写大模型裁判声明式接口；针对内容保真度、可读性、结构完整性三大指标，编写严谨防偏置的 System Prompt；利用结构化输出机制约束大模型直接将评估结果格式化映射为 Java 实体类。

- **人机对齐度量模块 (PearsonCalculator)**: 在 Java 中手动实现皮尔森相关系数数学公式。编写后台校准运行服务逻辑, 在输入人工评分黄金集后, 能够自动运行回归测试, 动态算出裁判打分与人类打分趋势的一致性相关系数。
- **标定黄金数据集**: 整理并人工标定 10 组涵盖“误改语义、过度脱敏、残缺表格、标题断裂”等多种典型清洗噪声的数据集, 作为系统的自校准与回归测试基线。

3. 由靖喆—— 研发任务包

主要负责用户交互、联调测试、日常进度跟进与结题技术文档整理。具体任务包包括:

- **可视化质检看板 (Dashboard)**: 负责前端单页质检仪表盘的整体布局设计 (HTML5 + Tailwind CSS), 构建简洁美观的现代化前端页面。
- **数据映射与高亮**: 使用 JavaScript 将后端返回的缺陷列表中的原文/清洗文对应关系, 动态高亮标记到双栏文本区中; 集成 Chart.js, 根据后端返回的评估分数, 动态绘制保真度、可读性、结构分的三维质量雷达图。
- **测试协助与文档**: 协助组长和组员 A 进行系统的 API 边界测试、并发测试与性能测试; 负责记录每日研发日志, 整理并撰写最终的项目交付技术报告。

二、选题原因与立项背景

2.1 选题原因: 直面非结构化数据治理的核心痛点与前沿技术挑战

2.1.1 顺应非结构化数据治理与大模型语料质检的前沿趋势

在企业知识库构建 (RAG 检索增强生成) 和大模型微调 (SFT) 的生产管线中, “高质量、高干净度”的非结构化数据是决定模型最终表现的绝对核心。原始积累的 PDF、扫描件文本噪声极多, 通常需要经过复杂的去噪、修复和格式化“改写”。这些改写行为 (如去除水印、敏感信息脱敏、缩写规范化、字段映射) 极易引入**“过度清洗”和“语义误改”**的隐性风险。例如, 一份财务报表在清洗时被不小心删除了一行关键利润数据, 直接会导致下游 RAG 检索出毁灭性的错误结论。如何在流水线中自动、精准地发现这些“语义失真点”, 是当前行业大模型应用落地的核心卡点。

2.1.2 传统规则质检在柔性语义评估上的彻底失效

传统的基于字符串正则、断言或精确匹配的传统质检方案, 面对非结构化文本的柔性语义改写完全无能为力。比如“国家电网”被清洗后统一归一化为“国家电网有限公司”, 从字面上看这属于“变动”, 但从语义上看它完全保真。传统工具会将其误判为“篡改”。只有引入具备语义理解能力的大模型作为裁判, 才能胜任这种柔性语义质量评估任务。

2.1.3 攻克 LLM-as-Judge 范式落地中的三大硬核痛点

大模型裁判 (LLM-as-Judge) 是当前学术界与工业界最前沿的语义质检范式。然而, 大模型作为裁判在真实工程落地中, 面临着三大公认痛点:

1. **评分稳定性差**: 大模型存在推理随机性, 同一个文本在不同时间可能会判出完全不同的分数;
2. **黑盒评估无解释性**: 通常只输出一个综合分, 无法给出具体的“瑕疵定位锚点”;
3. **计算资源极其昂贵**: 直接将万字长文档投喂给大模型做逐字比对, Token 消耗巨大, 且存在严重的“长文本偏置 (Lost in the Middle)”。本课题选择该选题, 能够让我们深入探索并攻克这些行业硬核痛点, 技术含量高, 极具挑战性。

2.2 对课题题目与核心任务的深层理解

通过深入研读任务书，我们团队对本课题的本质、设计哲学及核心目标形成了如下深层次的理解：

2.2.1 对“智能体（Agent）”服务化定位的理解

本课题研发的不是一个临时运行的文本处理脚本，而是一个**高内聚、轻量级、暴露标准 HTTP API、支持注册制集成**的“质检安全网关”。在企业级数据治理流水线中，它扮演着“无侵入质量过滤器”的角色。它本身绝不改写或修改任何业务数据，只负责对比“治理前文本”与“治理后文本”，对数据质量进行客观打分，精准指出缺陷原因，并将评估结果回传给流程编排器，从而驱动数据的“自动入库”、“打回重跑”或“降级转人工复核”等精细化流程路由。

2.2.2 对“Hybrid AI（混合智能）”设计哲学的理解：规则前置，模型兜底

任务书指出，“能用确定性规则解决就不消耗大模型算力”是系统设计的底层逻辑。如果盲目将整篇长文档直接投喂给大模型进行语义比对，大模型会因为位置偏置而忽略段落中间的错误，且 Token 算力开销和时间时延将极其高昂，不具备工程落地可行性。我们对本课题的突破思路是：利用 Myers 差分对齐引擎将长文本降维，先用毫秒级响应的 Java 规则锁死有改动或删除的“可疑句子对”。只有这些可疑的差异行才会被送审大模型裁判。这极大地降低了 80% 以上的大模型推理成本，缩短了响应时延，并确保瑕疵定位可以精准到句级和字符级。

2.2.3 对“稳、准、可复现”三大核心质量指标的理解

- **关于“稳”的理解（方差平抑与多裁判投票）**：大模型裁判天然具有不确定性。我们理解，“稳”的本质是通过工程手段平抑判罚方差。系统在 Java 后端通过线程池异步并发调用 3 次判罚，采用中位数求分、多数裁判票决（Majority Voting）通过瑕疵的共识机制，彻底消除大模型的幻觉及随机性，实现低温度下的可复现。
- **关于“准”的理解（人机一致性度量）**：“准”不能靠主观臆断。在开题阶段，我们就规划在系统中内置皮尔森相关系数计算器，自动运行“大模型评分”与“人工标准专家评分”的回归校准。只要我们能证明两者的相关系数 $r \geq 0.85$ ，就证明该智能体具备了在生产线上替代人工质检的科学性与置信度。
- **关于“可解释（瑕疵可定位）”的理解**：裁判给出的不只是一个分数，还必须精准输出包含“原句子 → 治理后句子 → 扣分理由”的锚点级瑕疵清单（定位准确率 $\geq 90\%$ ），这要求前端能够高亮渲染对应的变动句子，提供像素级的追溯力。
- **关于“可复现、可追溯”的理解（trace_id 全链路追溯）**：系统要求对改动“过程可审计”。智能体接收到待评测任务后，必须自动注入并透传唯一的 `trace_id`，将每一次大模型裁判调用的 Prompt 详情、中间投票概率、最终判罚结论完整落库。任何质量争议均可通过 `trace_id` 调出原始质检轨迹进行复盘与审计。

三、系统核心模块设计（展示系统的大容量与复杂度）

为了支撑本项目的“大系统”体量，项目被划分为以下五个互相解耦、通过标准契约交互的核心模块：

3.1 文本分句与 Myers 差分对齐模块（MyersTextAligner）

- **工作机制**：负责读取原文和清洗文，利用正则表达式进行句级划分（以句号、问号、叹号等结束符为边界）。

- **算法实现**：引入 `java-diff-utils`。使用经典的 Myers 差分编辑距离算法（时间复杂度 $O(ND)$ ），比对两个句子列表的差异。将 `DELETE`（被删除的句子）与 `CHANGE`（被修改的句子）提取出来作为“可疑句对”，并分配对应的物理锚点（源页码、段落索引）。

3.2 声明式大模型裁判模块（LLMJudgeAgent）

- **技术实现**：基于 LangChain4j 的 `AiServices` 模式。定义强类型接口，配置大模型 API 客户端（配置本地离线运行的开源模型或商用大模型 API）。
- **Prompt 工程**：设计带有严格 JSON 约束（JSON Schema）的 System Message。输入特定的“可疑句子对”，要求裁判仅判定：“该变动是否属于过度清洗？保真度扣除分值是多少？判定理由是什么？”。利用强类型映射，直接将大模型返回的 JSON 转换为 `domain.FlawItem` 实体对象。

3.3 并发多裁判协同决策引擎（VotingStableEvaluator）

- **并发设计**：由于大模型裁判推理耗时较长，系统在 Java 层面使用自定义配置的线程池。当一次评估请求进来时，系统异步发起 3 个独立的裁判任务。
- **共识融合**：
 - **分值平抑**：保真度得分、可读性得分去掉最大、最小值求平均。
 - **瑕疵多数决（Majority Voting）**：利用哈希映射（HashMap）统计瑕疵。只有当 ≥ 2 个裁判在相同的源句锚点上都判定了保真度丢失，该缺陷才被正式采纳并记录到瑕疵清单中，有效拦截单次大模型产生的评分随机性与偶发幻觉。

3.4 人机一致度校准模块（PearsonCalculator）

- **工作机制**：在数据库中内置 10 组黄金标准测试对（涵盖不同清洗噪声类型）以及人工专家的评分。
- **数学算法**：不调用任何第三方黑盒包，在 `PearsonCalculator.java` 中手写经典的皮尔森积矩相关系数公式，计算裁判模型在黄金集上的表现，输出定量的一致性评估指标，确保质检网关的可信任度。
- **计算公式**：

$$r = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum (X_i - \bar{X})^2 \sum (Y_i - \bar{Y})^2}}$$

3.5 交互式前端质检看板（index.html）

- **视觉交互**：使用 Tailwind CSS 设计现代化的双栏布局。左边展示原文，右边展示治理文。
- **高亮高逼真渲染**：前端 Axios 接收到 `EvaluationReport` 后，将瑕疵列表（`Flaws`）中的原文/清洗文对应关系，动态高亮标记到右栏文本中。并且，利用 JavaScript 悬停触发机制，当鼠标指针停留在带有红/黄色高亮背景的争议句子上时，动态浮现气泡卡片，清晰展现缺陷的具体类型（过度清洗/语义篡改）、判罚严重程度及详细的裁判扣分理由。
- **图表展现**：引入 Chart.js，根据后端返回的维度分绘制直观的三维质量雷达图，并提供一键触发“人机一致性度量标定”的交互按钮，展示校准曲线。